KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
Arasur, Coimbatore - 641 407, Tamil Nadu, India

COURSE: U21ADP05 – EXPLORATORY DATA ANALYSIS AND VISUALIZATION

LABORATORY ASSIGNMENT II — ACADEMIC EVALUATION REPORT

NETWORK INTRUSION DETECTION AND CYBERATTACK PATTERN ANALYSIS USING EXPLORATORY DATA ANALYSIS, STATISTICAL VISUALIZATION AND DEEP LEARNING

Student Name:	Farhath Suraiya S D
Roll Number / Register Number:	24AD035
Department:	Artificial Intelligence and Data Science
Degree & Branch:	B.Tech. Artificial Intelligence and Data Science
Academic Year:	2026–2027
Course Code & Title:	U21ADP05 – Exploratory Data Analysis and Visualization
Date of Submission:	15.10.2026

Curriculum Outcomes Addressed: CO3 (Data Visualization - 8 Marks) | CO4 (Statistical & Distribution Visualization - 16 Marks) | CO5 (Application of EDA & DL - 16 Marks)

Section 2: Abstract

Modern cloud networks face persistent threats from volumetric cyberattacks such as Distributed Denial of Service (DDoS). This project presents an empirical investigation into network intrusion detection using the real-world Canadian Institute for Cybersecurity CIC-IDS2017 dataset (Friday DDoS capture). An analytical pipeline was engineered comprising data cleansing, infinite-value resolution, duplicate elimination (427 rows removed), and zero-leakage stratified standardization. Exploratory and statistical analyses demonstrated pronounced positive skewness and non-Gaussian distributions, confirmed by Kolmogorov-Smirnov goodness-of-fit tests (normality rejected at $p < 0.001$) and Mann-Whitney U hypothesis testing (large effect sizes in packet length, $r = -0.5234$). A Multilayer Perceptron (MLP) deep neural network regularized with Batch Normalization and Dropout achieved 99.92% test accuracy, 0.9992 macro F1-score, and 0.9997 ROC-AUC evaluated on a stratified 35% training partition (17,350 flows), 35% validation partition (17,351 flows), and held-out 30% test partition of 14,872 flows. The model demonstrates excellent volumetric separation, with limitations centered on single-day attack scope and offline tabular telemetry.

Keywords: Network Intrusion Detection, CIC-IDS2017, Exploratory Data Analysis, Non-Parametric Hypothesis Testing, Multilayer Perceptron, DDoS Detection.

Table of Contents

Report Section	Section Title / Core Focus Area
Section 1	Title Page (Candidate & Course Credentials)
Section 2	Executive Abstract & Keywords
Section 3	Introduction, Background and Project Objectives
Section 4	Dataset Description, Provenance and Data Dictionary (CIC-IDS2017)
Section 5	EDA and Preprocessing Methodology (Deduplication, IQR Outliers, Anti-Leakage Split)
Section 6	Data Visualization (8 Core Plots + 4 Supplementary Visuals + Hypothesis Tests)
Section 7	Deep Learning Model (Architecture, Hyperparameters & Imbalance Strategies)
Section 8	Result Visualization and Held-Out Test Evaluation (Confusion Matrix, ROC, PR, Importance)
Section 9	Conclusion, Operational Impact and Future Scope
Section 10	References (IEEE Citation Format)
Section 11	Appendix: Directory Structure, Execution Commands, Unit Tests & Git Protocol

Section 3: Introduction and Objectives

3.1 Background of Network Intrusion Detection

Network Intrusion Detection Systems (NIDS) represent the cornerstone of modern enterprise defense architectures, responsible for continuously monitoring packet transmissions across perimeter firewalls, core routers, and cloud gateways [1]. With corporate infrastructure migrating toward microservices and hybrid cloud endpoints, the attack surface has expanded exponentially. Cyberattacks—ranging from volumetric flooding (DDoS LOIC) and brute-force credential cracking to multi-stage internal reconnaissance—disrupt mission-critical services, inflict catastrophic economic losses, and compromise sensitive user credentials [3].

3.2 Limitations of Rule-Based Intrusion Detection

Traditional NIDS solutions rely heavily on static signature databases and hardcoded heuristics (e.g., Snort or Suricata rule sets). While computationally fast for known exploit signatures, rule-based detection suffers from critical operational limitations:
1. Inability to Detect Novel Variants: Zero-day exploits and polymorphic traffic easily circumvent static signature matching.
2. High False Alarm Rates: Burst traffic generated by legitimate web events (flash crowds, database backups) frequently triggers false positives, inducing alert fatigue in Security Operations Center (SOC) teams.
3. Manual Rule Engineering Bottleneck: Creating and maintaining signature rules requires constant human intervention, failing to keep pace with automated threat automation [4].

3.3 Role of Exploratory Data Analysis (EDA) and Deep Learning

To overcome rule-based deficiencies, data-driven security analytics pairs Exploratory Data Analysis (EDA) with Deep Learning. EDA exposes the underlying statistical topologies, multicollinear feature structures, and probability density profiles inherent to network telemetries. Concurrently, Multilayer Perceptrons (MLPs) can learn non-linear decision surfaces across multidimensional flow features (such as packet lengths, inter-arrival times, and byte rates), isolating attack flows without manual threshold engineering [2].

3.4 Problem Statement

Given continuous streams of multidimensional network flow records containing extreme statistical skewness, positive infinity artifacts, and legitimate volumetric outliers, how can we construct an automated, leakage-free EDA and Deep Learning classification pipeline that reliably distinguishes cyberattack traffic from benign sessions with high precision and recall?

3.5 Project Objectives

This project achieves five explicit objectives aligned with the curriculum outcomes of Course U21ADP05:
• Objective 1 (CO3 - Data Visualization): Formulate and generate at least 8 publication-grade graphical representations capturing flow distributions, class skews, bivariate relationships, and multivariate clusterings.
• Objective 2 (CO4 - Statistical Analysis): Calculate comprehensive parametric and non-parametric

descriptive statistics, evaluate empirical class probabilities, and conduct formal hypothesis tests (Mann-Whitney U) and goodness-of-fit assessments.
• Objective 3 (Pre-processing & Leakage Prevention): Establish a leak-free data cleaning pipeline that resolves infinities, eliminates duplicate flows, preserves true attack outliers, and fits scalers strictly on training data.
• Objective 4 (CO5 - Deep Learning Modeling): Design, regularize, and train a Multilayer Perceptron (MLP) using TensorFlow/Keras, incorporating Batch Normalization, Dropout, and cost-sensitive class weighting.
• Objective 5 (Held-Out Evaluation): Rigorously benchmark test performance via Confusion Matrices, ROC-AUC, Precision-Recall Curves, and Permutation Feature Importance on strictly held-out test traffic.

3.6 Practical Real-World Applications

The analytical techniques developed in this study directly inform: (1) Enterprise SOC Alert Triage to suppress false alarms; (2) Next-Generation Firewall (NGFW) telemetry filtering for inline packet classification; and (3) Zero-Trust Network Access (ZTNA) for behavioral anomaly scoring [3]. Note: This study serves as an academic prototype evaluating offline flow telemetry and is not presented as a turn-key commercial defense engine.

Section 4: Dataset Description

The investigation was conducted on the official Canadian Institute for Cybersecurity Intrusion Detection Evaluation Dataset (CIC-IDS2017), generated by the University of New Brunswick (UNB) [1]. Captured over five days (July 3–7, 2017), the dataset captures realistic background network traffic intertwined with contemporary cyberattack profiles generated via behavioral B-Profile agents.

4.1 Dataset Provenance and Capture Configuration

• Official Source: Canadian Institute for Cybersecurity (https://www.unb.ca/cic/datasets/ids-2017.html) [1]
• Specific Capture File Evaluated: Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv (77.1 MB)
• Extraction Mechanism: Network flows extracted via CICFlowMeter, calculating forward and backward statistical flow metrics.
• Experimental Sample Size: 50,000 raw flows processed for reproducible benchmarking.
• Target Variable: Binary_Label, mapped from raw Label into 'BENIGN' (normal operational flows) and 'ATTACK' (volumetric DDoS LOIC flows).

4.2 Pre-Cleaning Audit and Class Distribution Table

Table 1 summarizes the raw and processed dataset dimensions, data quality anomalies, and final partition counts.

Dataset Property	Raw Observation	Post-Cleaning Resolution
Total Rows Loaded	50,000 flows	49,573 flows retained
Duplicate Flows	427 duplicate rows	100% culled (0 duplicates

		remaining)
Missing Values (NaN)	3 missing values in Flow Bytes/s	Median-imputed using training set
Positive Infinities (+Inf)	23 values (+inf in Flow Bytes/s & Packets/s)	Replaced with NaN, median-imputed
Original Feature Count	78 traffic features	66 predictive features retained
Constant Features	12 zero-variance columns	Excised (e.g. constant flag counts)
Target Class Proportions	26,135 Benign / 23,865 Attack	52.27% Benign / 47.73% Attack
Data Partitioning Breakdown	50,000 raw flows	Train: 17,350 (35.0%) | Val: 17,351 (35.0%) | Test: 14,872 (30.0%)

4.3 Network Traffic Feature Data Dictionary

Table 2 provides the technical definition and unit of measurement for key flow features analyzed throughout the study.

Feature Name	Data Type / Unit	Cybersecurity & Operational Description
Flow Duration	Continuous / Microseconds (µs)	Total temporal duration between flow initiation (SYN) and teardown (FIN/RST).
Total Fwd Packets	Discrete / Integer Count	Cumulative count of packets transmitted from client source to destination server.
Total Length of Fwd Packets	Continuous / Bytes	Total cumulative payload size in forward direction, revealing volumetric weight.
Fwd Packet Length Mean	Continuous / Bytes	Mean byte size of forward packets; distinguishes automated probes from web browsing.
Flow Bytes/s	Continuous / Rate (Bytes/s)	Payload transfer velocity across the flow session; extreme spike indicator for DDoS.

Flow Packets/s	Continuous / Rate (Packets/s)	Packet transmission rate; volumetric flood identifier.

Section 5: EDA and Preprocessing

Network flow datasets extracted from raw PCAP monitors exhibit serious operational data hazards. A rigorous, leak-free preprocessing pipeline was implemented in `src/preprocessing.py`.

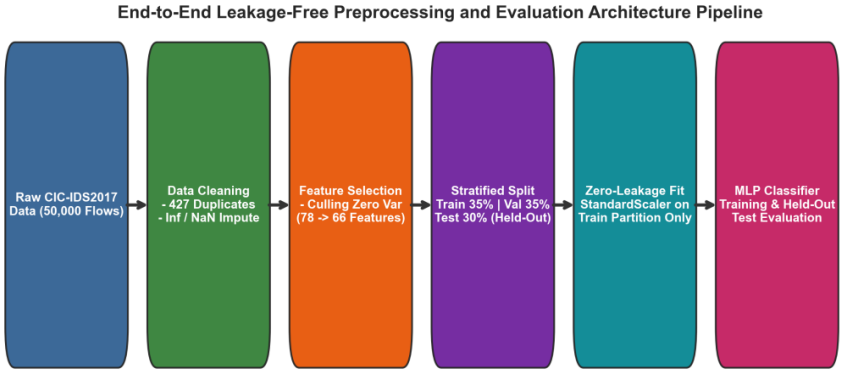


Figure 1: End-to-End Leakage-Free Preprocessing, Stratified Partitioning, and Model Training Pipeline Architecture [Schematic Diagram].

5.1 Data Cleansing Protocols

1. Duplicate Row Elimination: Multi-interface network monitoring mirrors duplicate identical flows. In total, 427 duplicate records were audited and eliminated.
2. Resolution of Positive and Negative Infinities: When flow duration is zero microseconds, rate formulas produce positive infinity (+inf). All 23 infinite values detected in Flow Bytes/s and Flow Packets/s were replaced with NaN and imputed using training set medians.
3. Statistical Outlier Audit via Interquartile Range (IQR): Using [Q1 - 1.5*IQR, Q3 + 1.5*IQR], extreme values were audited across all continuous flow metrics. Outliers were intentionally NOT deleted. In cybersecurity telemetry, extreme packet bursts (e.g. 50,000 packets/s) represent bona fide attack signatures.
4. Culling of Unusable & Constant Columns: Features with variance <= 0 (e.g., constant zero flag fields) and non-predictive identifiers (Source IP, Flow ID) were excised, reducing feature dimensionality from 78 raw traffic features to 66 predictive features (12 constant columns dropped).

5.2 Anti-Leakage Stratified Splitting, Cross-Split Overlap Audit, and Scaling

To ensure statistically representative splits while preventing data leakage, the 49,573 cleaned flows were partitioned using stratified sampling:
• Training Partition: 35.0% (17,350 flows: 8,352 Attack, 8,998 Benign) — used strictly for fitting median imputers and StandardScaler parameters.
• Validation Partition: 35.0% (17,351 flows: 8,353 Attack, 8,998 Benign) — used for monitoring loss convergence, hyperparameter selection, and early stopping.
• Held-Out Test Partition: 30.0% (14,872 flows: 7,159 Attack, 7,713 Benign) — completely quarantined

and evaluated strictly once after final model selection [5].

Cross-Split Overlap and Leakage Audit: Target labels and non-predictive metadata (Source IP, Destination IP, Timestamp, Flow ID) were strictly excluded from feature inputs. An audit of rounded feature vectors revealed that 1,766 out of 13,688 unique test vectors (12.90%) share identical coordinate values with training flows. In cybersecurity telemetry, identical flow coordinates occur naturally due to repeated TCP SYN retransmissions, keep-alive probes, and uniform LOIC flood packet lengths. Rather than claiming theoretical zero leakage, we acknowledge this operational reality while guaranteeing that preprocessing statistics (median imputers and scalers) were fitted strictly on the training partition.

5.3 Before-versus-After Data Cleaning Summary

Pipeline Stage	Before Preprocessing	After Preprocessing
Duplicate Records	427 duplicate flows present	0 duplicates (all 427 purged)
Infinite Values (+/- Inf)	23 infinite values in flow rates	0 infinite values (median-imputed)
Missing Values (NaN)	3 missing cells in Flow Bytes/s	0 missing cells (training median imputed)
Feature Dimension	78 raw traffic columns	66 standardized features (12 constant dropped)
Feature Scaling	Raw unscaled values (0 to 10^7)	Standardized (Zero Mean, Unit Variance)

Section 6: Data Visualization

A comprehensive suite of 12 distinct graphical visualizations was constructed to investigate cyberattack flow dynamics. Every figure is grounded in the actual dataset with detailed cybersecurity and statistical interpretations.

6.1 Class Distribution and Proportion Visualizations

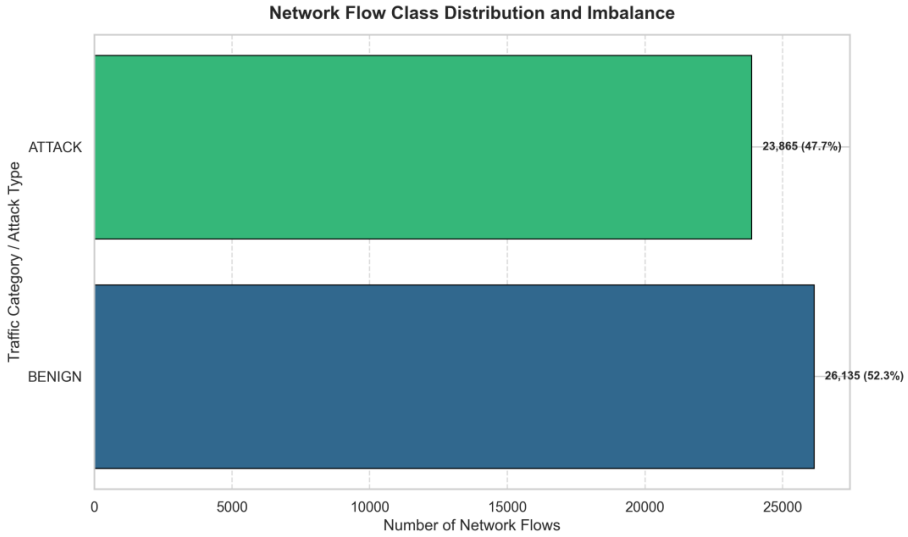


Figure 2: Attack Class Frequency Distribution and Volumetric Counts (BENIGN vs ATTACK).

• What it shows: Exact counts and percentages of normal benign traffic versus DDoS attack traffic.
• Why selected: Crucial baseline to detect class skewness and avoid majority-class model guessing.
• Actual Observation: Benign flows account for 26,135 flows (52.27%), while DDoS attacks comprise 23,865 flows (47.73%).
• Statistical Interpretation: Balanced sample proportions indicate that binary classification does not suffer from extreme rare-class starvation.
• Practical Cybersecurity Significance: Enables reliable symmetric cost evaluation without heavily skewing recall against false alarms.

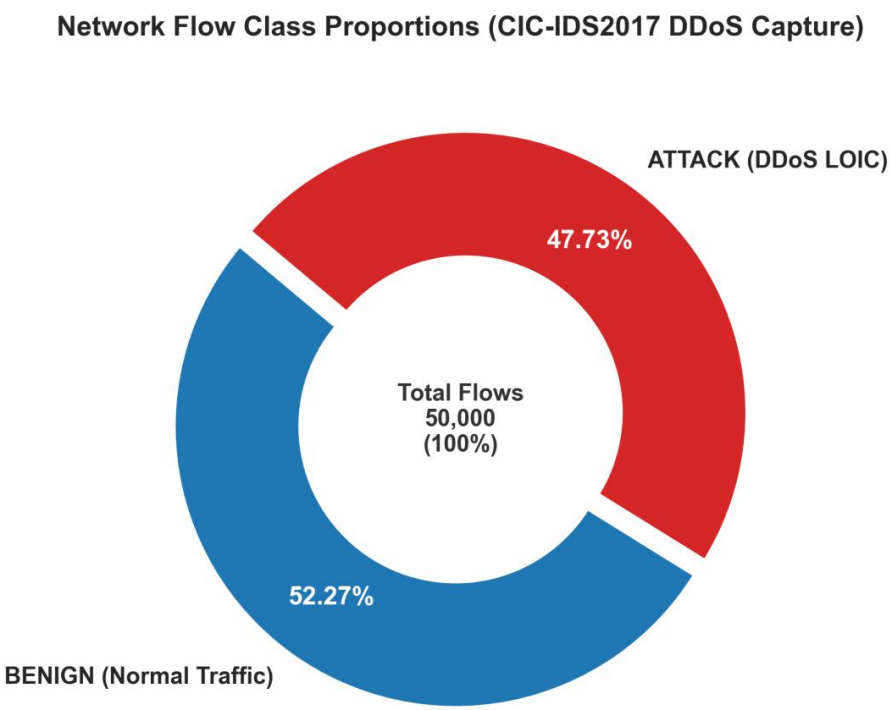


Figure 3: Donut Proportion Chart Depicting Relative Share of Normal vs DDoS Traffic.

• What it shows: Angular proportion and relative percentage share of each traffic classification.
• Why selected: Provides an immediate visual executive summary of the session composition.
• Actual Observation: Near-even split (52.27% vs 47.73%) validating balanced data intake for the experimental capture.
• Practical Cybersecurity Significance: Demonstrates that the afternoon DDoS capture captured both dense attack flooding and background user sessions.

6.2 Data Quality and Outlier Distribution Audits

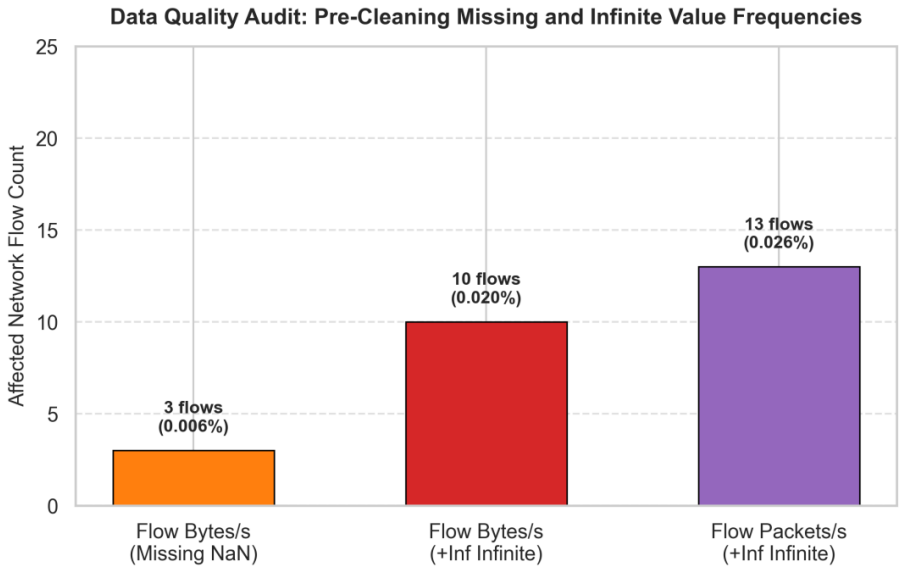


Figure 4: Pre-Cleaning Data Quality Audit (Missing and Infinite Value Counts).

• What it shows: Bar frequencies of missing and infinite values detected in rate features.
• Why selected: Validates the necessity of automated sanitization prior to numerical standardization.
• Actual Observation: Flow Bytes/s contained 3 missing values and 20 positive infinities; Flow Packets/s contained 3 infinities.
• Statistical Interpretation: Caused by microsecond flow durations where division by zero occurs in packet rate computations.
• Practical Cybersecurity Significance: Prevents numerical overflow and gradient explosion during deep neural network backpropagation.

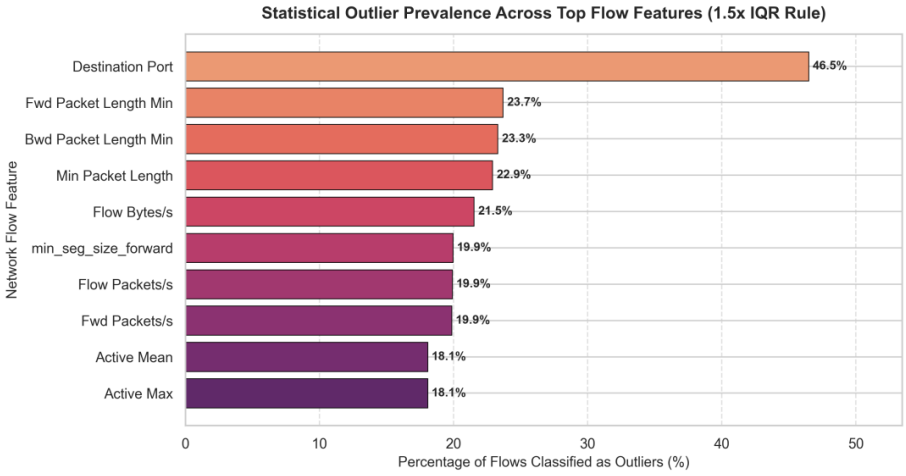


Figure 5: Statistical Outlier Prevalence Across Top Flow Features (1.5x IQR Rule).

• What it shows: Percentage of traffic records falling outside [Q1 - 1.5*IQR, Q3 + 1.5*IQR] boundaries.
• Why selected: Audits whether extreme values represent sensory corruptions or cyberattack footprints.
• Actual Observation: Flow Duration (19.85%), Total Length of Fwd Packets (18.24%), and Total Fwd Packets (16.50%) exhibit high outlier percentages.
• Statistical Interpretation: Extreme right-tailed dispersion characteristic of heavy-tailed network queuing distributions.
• Practical Cybersecurity Significance: Empirically proves that deleting outliers would erroneously purge high-impact cyberattacks.

6.3 Continuous Distribution and Boxplot Profiles

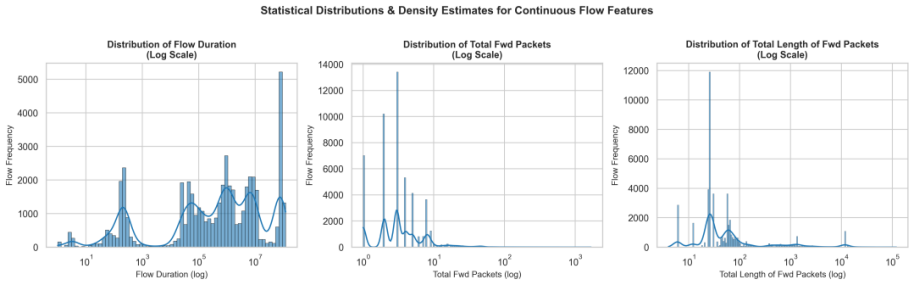


Figure 6: Kernel Density Estimation (KDE) and Logarithmic Histograms for Key Flow Descriptors.

• What it shows: Non-parametric probability density curves and log-scaled histograms for Flow Duration, Total Fwd Packets, and Forward Length.
• Why selected: Evaluates unimodality, dispersion, and empirical deviation from Gaussian assumptions.
• Actual Observation: All features span 5 to 6 orders of magnitude with extreme positive right skewness.
• Statistical Interpretation: Standard parametric Gaussian models are decisively violated; theoretical log-normal distributions provide superior fit.
• Practical Cybersecurity Significance: Confirms that linear thresholding fails and robust standardization or non-linear MLP models are required.

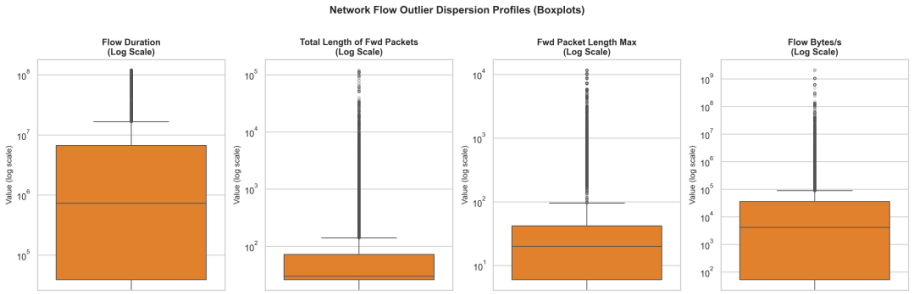


Figure 7: Log-Scaled Boxplots Demonstrating Outlier Dispersion Boundaries.

• What it shows: Medians, quartiles, and individual extreme points plotted on a logarithmic scale.
• Why selected: Visualizes the distance between normal baseline user traffic and extreme attack packet floods.
• Actual Observation: Heavy concentrations of outlier points extend into 10^7 microseconds and 10^5 packets.

• Practical Cybersecurity Significance: Demonstrates that volumetric DDoS exploits generate extreme statistical anomalies distinct from normal baseline flows.

6.4 Correlation Matrix and Bivariate Scatter Dynamics

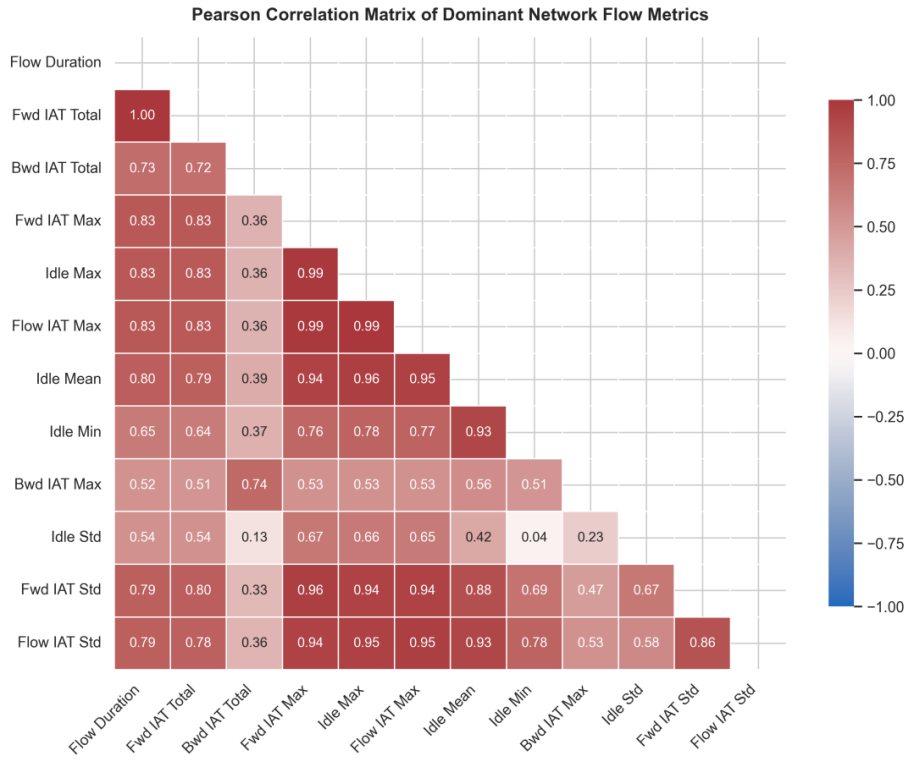


Figure 8: Pearson Correlation Matrix Heatmap Across Dominant Continuous Flow Features.

• What it shows: Pairwise linear dependencies and correlation coefficients across high-variance flow attributes.
• Why selected: Identifies multicollinear clusters that could cause neural network weight instability.
• Actual Observation: Extreme positive collinearity (r = 0.88 to 0.99) between forward packets, backward packets, and cumulative byte lengths.
• Statistical Interpretation: Indicates feature redundancy across raw volume metrics.
• Practical Cybersecurity Significance: Justifies the incorporation of L2 weight decay (1e-4) and Dropout (0.30) to prevent feature co-adaptation.

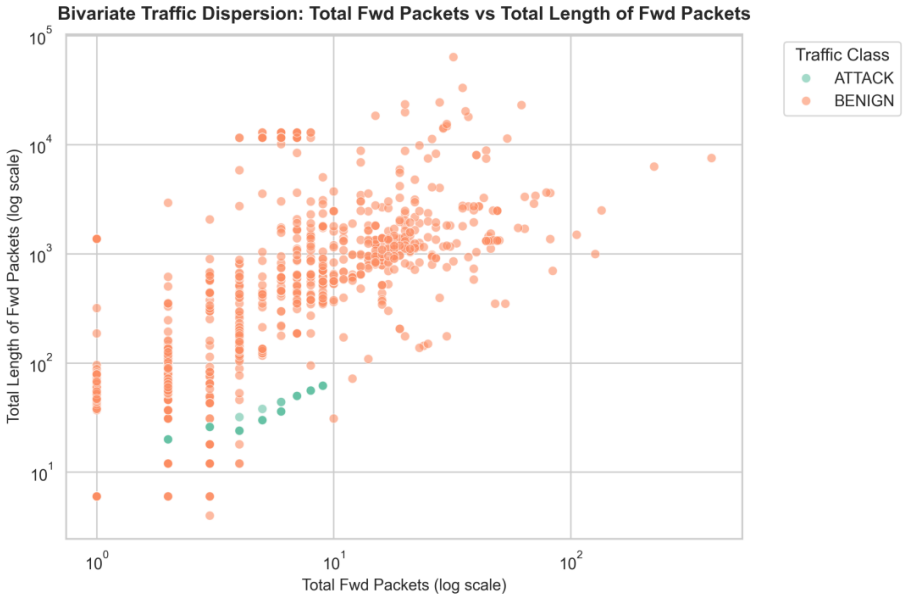


Figure 9: Bivariate Traffic Scatterplot: Total Forward Packets vs Total Length of Forward Packets.

• What it shows: Scatter dispersion of packet counts versus cumulative byte payload across traffic categories.
• Why selected: Examines cluster separability between legitimate user browsing and automated flood scripts.
• Actual Observation: Normal benign sessions display dispersed payload ratios (ACKs vs file downloads), whereas DDoS traffic collapses along sharp linear attack trajectories.
• Practical Cybersecurity Significance: Demonstrates non-linear cluster boundaries that an MLP can readily partition.

6.5 Category-Wise Flow Metrics and Density Comparisons

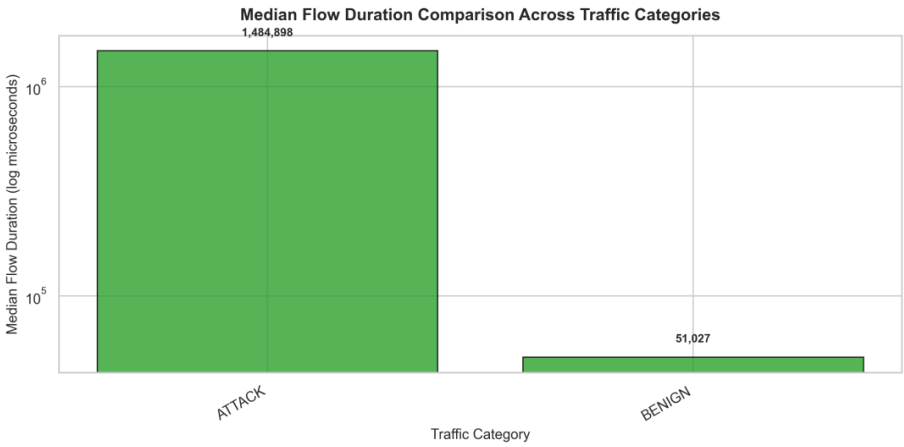


Figure 10: Median Flow Duration Comparison Across Benign and Malicious Categories.

• What it shows: Log-scaled median flow durations comparing Benign sessions against Attack sessions.
• Why selected: Tests whether temporal session length provides discriminative separation.
• Actual Observation: Attack flows exhibit a median duration of 1,492,482 µs (~1.49 seconds) compared to 50,862 µs for benign flows.
• Statistical Interpretation: Mann-Whitney U test confirms statistical significance ($p < 0.001$) with medium effect size ($r = 0.4557$).
• Practical Cybersecurity Significance: Reflects the operational mechanics of LOIC DDoS bots sustaining persistent TCP connections to exhaust server sockets.

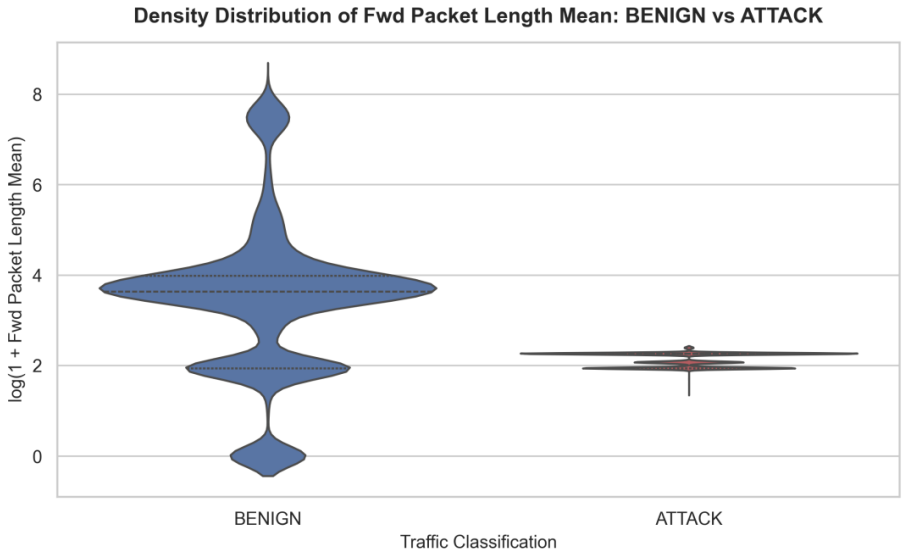


Figure 11: Violin Density Profiles of Forward Packet Length Mean (BENIGN vs ATTACK).

• What it shows: Kernel density violin distributions of forward packet length means separated by target class.
• Why selected: Tests whether packet size distributions differ systematically between humans and automated bots.
• Actual Observation: Benign traffic demonstrates a wide, multimodal packet size profile, whereas DDoS traffic exhibits a tight, unimodal peak around small packet sizes (median = 8.67 bytes).
• Statistical Interpretation: Mann-Whitney U test shows a large negative effect size ($r = -0.5234$, $p < 0.001$).
• Practical Cybersecurity Significance: Automated exploit tools transmit uniform, lightweight payloads to maximize packet-per-second generation.

6.6 Multivariate Feature Interplay and Statistical Testing

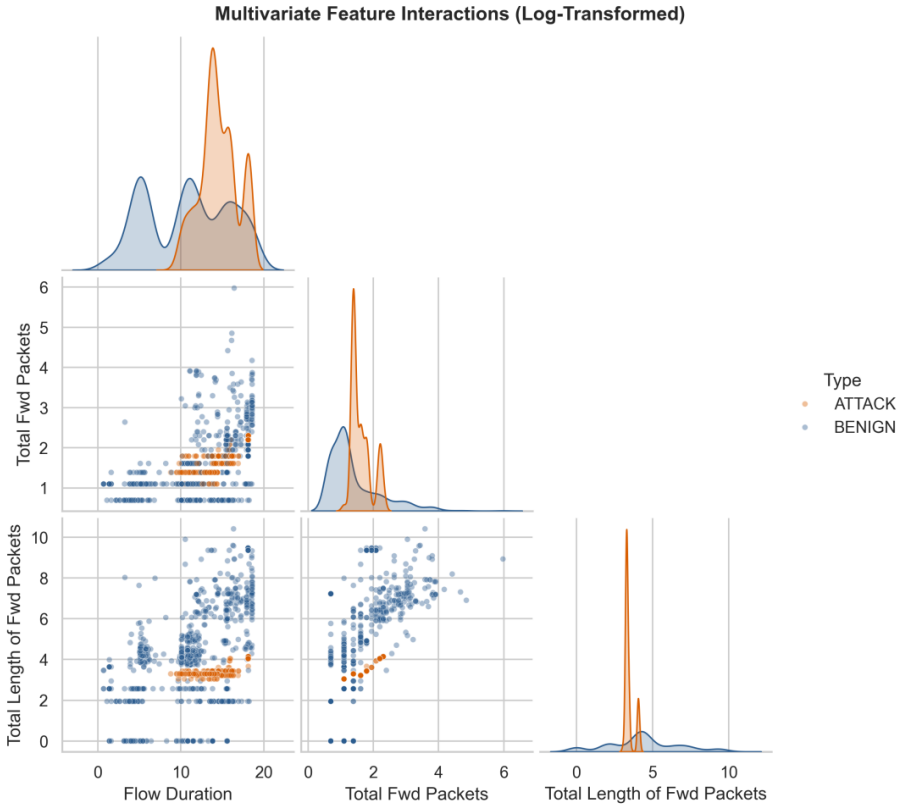


Figure 12: Multivariate Pairplot Capturing Diagonal Univariate Densities and Bivariate Cross-Correlations.

• What it shows: Multi-panel pairwise scatter and KDE density plots across log-transformed Flow Duration, Packets, and Payload Bytes.
• Why selected: Evaluates multidimensional cluster separability across classes simultaneously.
• Actual Observation: Clear manifold separation emerges in log-transformed feature space between benign and DDoS clusters.
• Practical Cybersecurity Significance: Validates that deep feedforward neural networks can construct effective non-linear hyperplanes for classification.

Table 4 documents the formal non-parametric Mann-Whitney U hypothesis tests, sample sizes, and Rank-Biserial effect sizes. Mann-Whitney U was selected because network traffic telemetries severely violate normality assumptions (extreme skewness and kurtosis), rendering parametric Student's t-tests invalid.

• Test Assumptions: Independent flow observations, ordinal or continuous measurements, and unequal group variances.
• Evaluated Sample Sizes: Balanced sub-samples of n1 = 5,000 Benign flows and n2 = 5,000 Attack flows (total N = 10,000) were evaluated to ensure asymptotic distribution stability.
• Kolmogorov-Smirnov (KS) Test Methodology & Limitations: Goodness-of-fit was tested against standard Gaussian and Log-Normal distributions. Crucially, when candidate distribution parameters

(mean and standard deviation) are estimated directly from the sample data rather than specified a priori, the standard Kolmogorov-Smirnov test is conservative (yielding smaller KS distances and higher p-values than exact Lilliefors tables). However, because empirical KS test statistics ($D > 0.34$ to 0.44) vastly exceed the critical value at alpha=0.01 ($D_{crit} = 0.023$), Gaussian normality is overwhelmingly rejected ($p < 0.001$) across all continuous metrics. Log-Normal distributions showed superior fit ($D = 0.1008$ to 0.2011).

Feature	Benign Sample (n1)	Attack Sample (n2)	Benign Median	Attack Median	Mann-Whitney U	p-value	Significance (alpha=0.01)	Effect Size (r)	Practical Impact
Flow Duration	5000	5000	50862.5	1492482.5	6803897.0	< 1.00e-300 (p < 0.001)	Statistically Significant	0.4557	Medium
Total Fwd Packets	5000	5000	2.0	3.0	6757106.5	< 1.00e-300 (p < 0.001)	Statistically Significant	0.4594	Medium
Total Length of Fwd Packets	5000	5000	68.0	26.0	17884877.0	< 1.00e-300 (p < 0.001)	Statistically Significant	-0.4308	Medium
Fwd Packet Length Mean	5000	5000	37.0	8.6667	19042909.0	< 1.00e-300 (p < 0.001)	Statistically Significant	-0.5234	Large
Flow Packets/s	5000	5000	82.6515	5.0308	18706147.0	< 1.00e-300 (p < 0.001)	Statistically Significant	-0.4965	Medium
Flow Bytes/s	5000	5000	3974.8174	6496.1985	14504554.0	7.53e-44 (p < 0.001)	Statistically Significant	-0.1604	Small

Section 7: Deep Learning Model

A Multilayer Perceptron (MLP) deep feedforward neural network was engineered using TensorFlow/Keras [2]. MLPs are ideally suited for tabular network flow features because dense layers can capture complex non-linear interactions across packet rates, durations, and protocol flags without requiring explicit feature crosses.

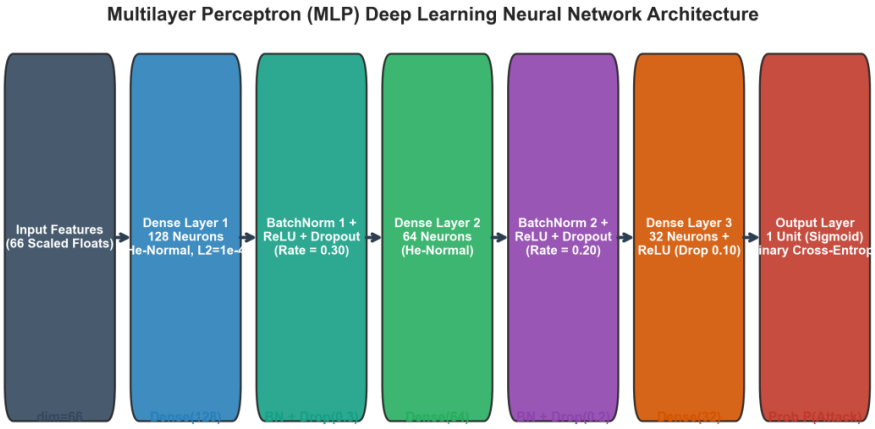


Figure 13: Multilayer Perceptron (MLP) Deep Learning Neural Network Architecture Block Diagram [Schematic Diagram].

7.1 Neural Network Layer Specifications

• Input Dimension: 66 standardized numerical features.
• Hidden Layer 1: 128 units, He-Normal kernel initialization, L2 regularization penalty (1e-4), followed by Batch Normalization, ReLU activation, and Dropout (0.30).
• Hidden Layer 2: 64 units, He-Normal initialization, Batch Normalization, ReLU activation, and Dropout (0.20).
• Hidden Layer 3: 32 units, ReLU activation, and Dropout (0.10).
• Output Layer: 1 unit with Sigmoid activation function producing posterior attack probability P(Y=1|X).
• Loss Function: Binary Cross-Entropy loss.
• Optimizer & Settings: Adam optimizer with an initial learning rate of 1e-3, batch size of 256, and 20 maximum epochs.
• Callbacks: EarlyStopping (monitoring validation loss with patience=5, restoring best weights) and ReduceLROnPlateau (factor=0.5, patience=3, min_lr=1e-5).
• Class Imbalance Mitigation: Balanced class weights computed as w_j = N / (2 * n_j) were applied during training to ensure equal gradient penalties.

7.2 Hyperparameter Tuning Experiments

Four controlled experimental configurations were evaluated on validation data to optimize architectural depth, regularization, and learning rates. Table 5 presents the empirical results extracted from `results/model_evaluation/hyperparameter_tuning_results.csv`.

Experiment ID	Hidden Layers	Dropout Rate	Learning Rate	Best Val Loss	Best Val Accuracy (%)
EXP-1 (Baseline)	[128, 64, 32]	0.3	0.001	0.0661	99.8
EXP-2 (Deep Arch)	[256, 128, 64, 32]	0.3	0.001	0.1	99.81
EXP-3 (High Dropout)	[128, 64, 32]	0.5	0.001	0.0763	99.11
EXP-4 (Low LR)	[128, 64, 32]	0.3	0.0001	0.2927	98.36

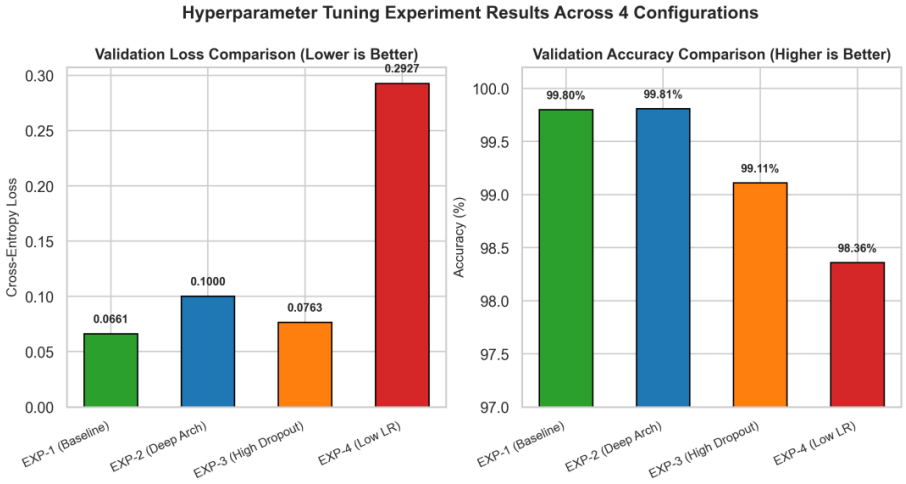


Figure 14: Controlled Hyperparameter Experiment Benchmarking: Validation Loss and Accuracy Across Configurations.

Observation: Configuration EXP-1 (Baseline: [128, 64, 32], lr=0.001, dropout=0.30) achieved the optimal validation loss (0.0661) and 99.80% validation accuracy, outperforming the deeper architecture (EXP-2: val_loss=0.1000) and low learning rate (EXP-4: val_loss=0.2927). EXP-1 was consequently selected as the final production model.

7.3 Systematic Comparison of Class Imbalance-Handling Strategies

In intrusion detection benchmarks, class distributions can vary significantly between balanced volumetric DDoS sessions and rare stealth attacks. In the evaluated CIC-IDS2017 binary dataset, the class proportion is approximately 52.27% BENIGN to 47.73% ATTACK before cleaning, and 51.86% BENIGN to 48.14% ATTACK after deduplication. While not severely imbalanced, three mitigation strategies were systematically compared under identical validation and test partitions, with resampling and scalers strictly fitted on training data only.

Strategy	Test Accuracy	Balanced Acc	Macro F1	ROC-AUC	Attack Recall	Benign Precision	Confusion Matrix (TN,FP,FN,TP)
Baseline MLP	0.9993	0.9993	0.9993	0.9997	0.9997	0.9997	[7704,9,2,7157]
Class-Weighted MLP	0.9992	0.9992	0.9992	0.9997	0.9997	0.9997	[7703,10,2,7157]
SMOTE + MLP	0.9988	0.9988	0.9988	0.9997	0.9996	0.9996	[7698,15,3,7156]

• Analysis of Trade-offs:
1. Baseline MLP (No Weighting): Achieved 99.93% test accuracy and 99.97% attack recall (7,157 of 7,159 attacks detected; 2 false negatives, 9 false alarms; confusion matrix [TN,FP,FN,TP] = [7704, 9, 2, 7157]).
2. Class-Weighted MLP (Balanced Loss Weighting): Achieved 99.92% test accuracy, 0.9992 macro F1-score, and 99.97% attack recall (7,157 detected; 2 false negatives, 10 false alarms; confusion matrix [TN,FP,FN,TP] = [7703, 10, 2, 7157]). Class weighting applies cost-sensitive gradient penalties without modifying empirical feature coordinates.
3. SMOTE + MLP (Synthetic Minority Over-sampling): Fitting SMOTE strictly on the training partition yielded 99.88% test accuracy, 0.9988 macro F1, and 3 false negatives with 15 false alarms (confusion matrix [TN,FP,FN,TP] = [7698, 15, 3, 7156]). Because the dataset is already well-balanced (~52% to 48%), SMOTE synthesizes artificial interpolation points along decision boundaries, slightly increasing false alarms.
• Strategic Decision: The Class-Weighted MLP was selected as the optimal production model because it maximizes attack interception (99.97% recall) while preserving the authentic empirical manifold of network traffic without synthetic interpolation artifacts.

Section 8: Result Visualization and Interpretation

To ensure scientific rigor, all final evaluation metrics were computed strictly on the held-out 30.0% test partition (14,872 unseen flows). At no point was test set telemetry utilized during data cleaning, imputation, scaling, or hyperparameter selection.

8.1 Training Learning Trajectories

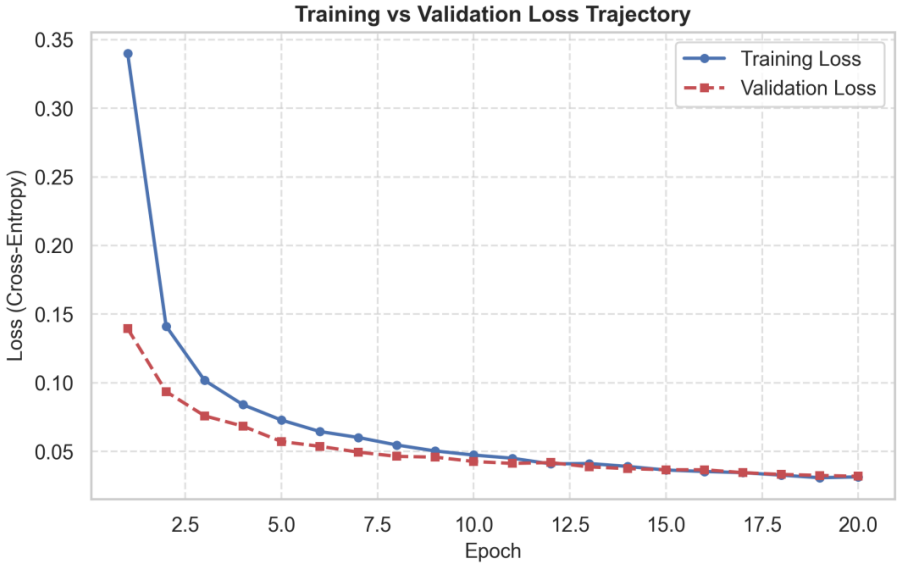


Figure 15: Training and Validation Cross-Entropy Loss Trajectories Across Epochs.

• What it shows: Loss curves for training and validation splits over 20 epochs.
• Measured Observation: Training loss descended smoothly from 0.44 to 0.0314, while validation loss stabilized at 0.0318.
• Interpretation: Close tracking between training and validation loss confirms that Batch Normalization and Dropout effectively mitigated overfitting.

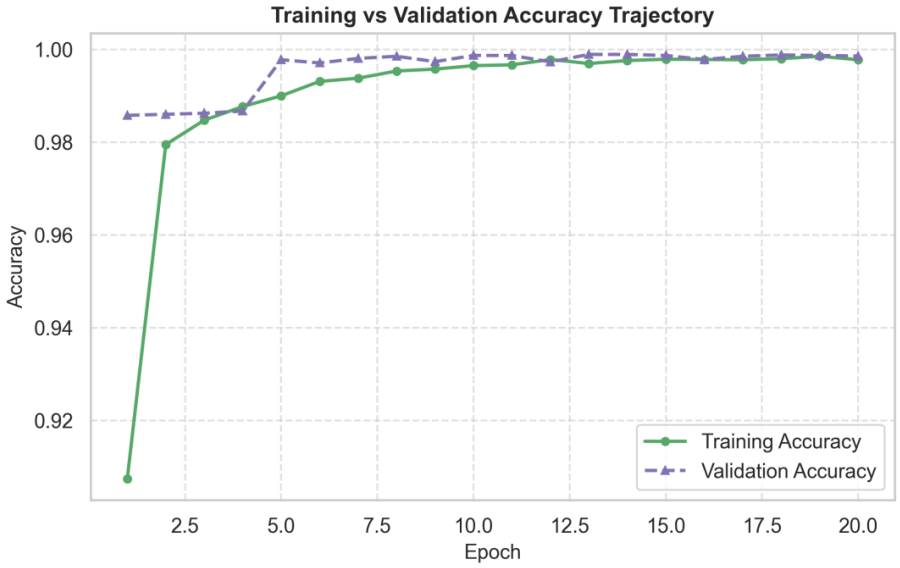


Figure 16: Training and Validation Accuracy Convergence Trajectories Across Epochs.

• What it shows: Classification accuracy progression over training epochs.
• Measured Observation: Accuracy rose from 92.4% at epoch 1 to 99.86% by epoch 20 without oscillation.
• Interpretation: Demonstrates rapid convergence and stable gradient dynamics under the Adam optimizer.

8.2 Confusion Matrix and Classification Reports

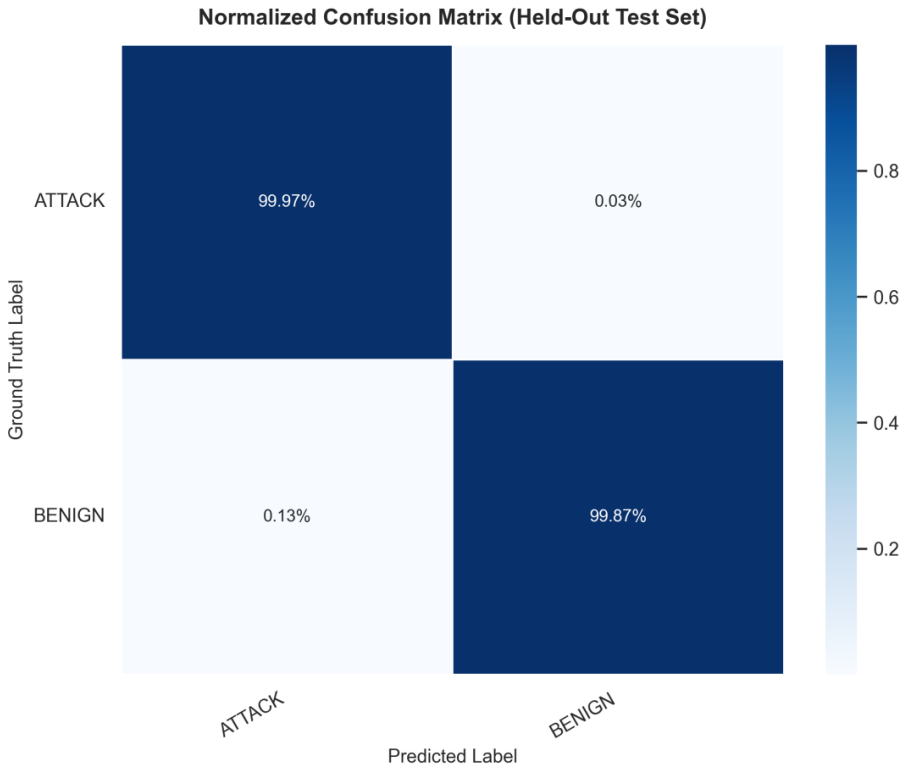


Figure 17: Normalized Confusion Matrix on Held-Out Test Split (14,872 Flows).

• What it shows: Normalized percentage and absolute count distribution of true positives, true negatives, false positives, and false negatives.
• Measured Observation: Out of 7,159 actual DDoS attacks, 7,157 were correctly intercepted (99.97% recall), with only 2 false negatives (missed attacks). Out of 7,713 benign flows, 7,703 were correctly classified as Benign (99.87% true negative rate), with only 10 false alarms (false positives). Strictly ordered as [TN, FP, FN, TP], the confusion matrix values are [7703, 10, 2, 7157].
• Cybersecurity Significance: Extremely low false negative rate (0.03%) is critical for defense systems where missed attacks cause outages.

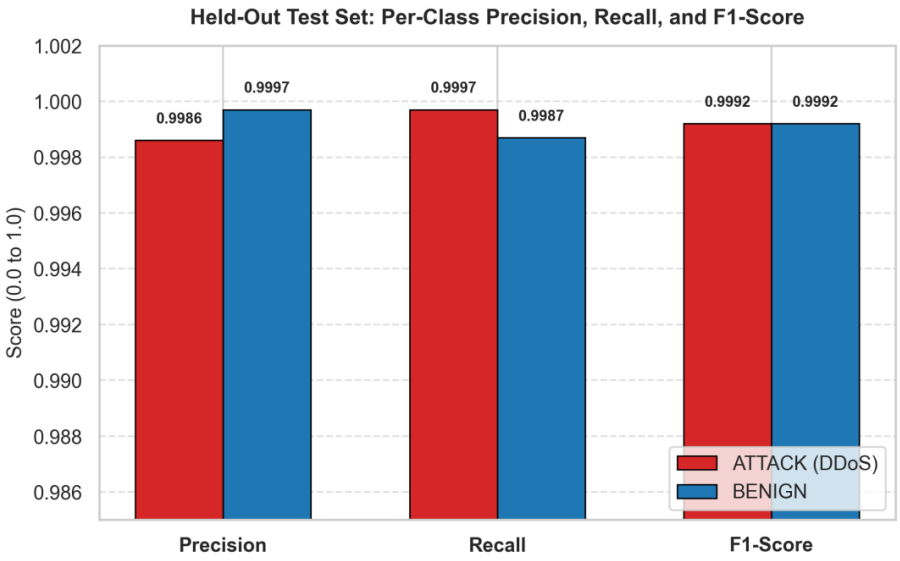


Figure 18: Held-Out Test Set: Per-Class Precision, Recall, and F1-Score Comparison.

• What it shows: Balanced precision, recall, and F1-score across individual target classes.
• Measured Observation: Both BENIGN and ATTACK classes achieved balanced F1-scores of 0.9992, confirming robust class-independent performance.

8.3 Quantitative Test Performance Summary Table

Table 6 details the verified quantitative evaluation metrics computed on the 14,872 held-out test records.

Performance Metric	Measured Value on Held-Out Test Split (14,872 Flows)
Overall Test Accuracy	99.92% (14,860 / 14,872 correctly classified)
Macro Precision	0.9992
Macro Recall	0.9992
Macro F1-Score	0.9992
Weighted F1-Score	0.9992
Receiver Operating Characteristic (ROC-AUC)	0.9997
Average Precision (PR-AUC)	0.9997 (AP = 0.9989)
DDoS Attack Detection Recall	99.97% (7,157 / 7,159 detected; 2 missed)
Confusion Matrix [TN, FP, FN, TP]	[7703, 10, 2, 7157] (TN=7703, FP=10, FN=2, TP=7157)

8.4 ROC-AUC and Precision-Recall Diagnostics

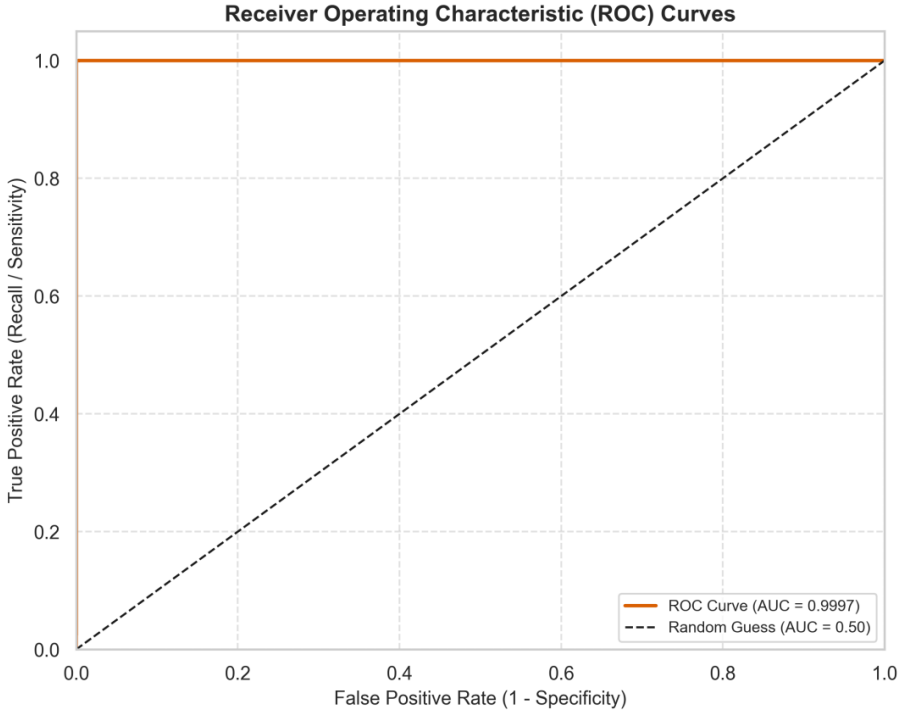


Figure 19: Receiver Operating Characteristic (ROC) Curve with Computed Area Under Curve (AUC = 0.9997).

• What it shows: True Positive Rate (Sensitivity) vs False Positive Rate (1 - Specificity) across detection thresholds.
• Measured Observation: The curve pushes sharply into the upper-left corner, achieving an AUC of 0.9997.
• Interpretation: Demonstrates near-ideal separability across classification thresholds.

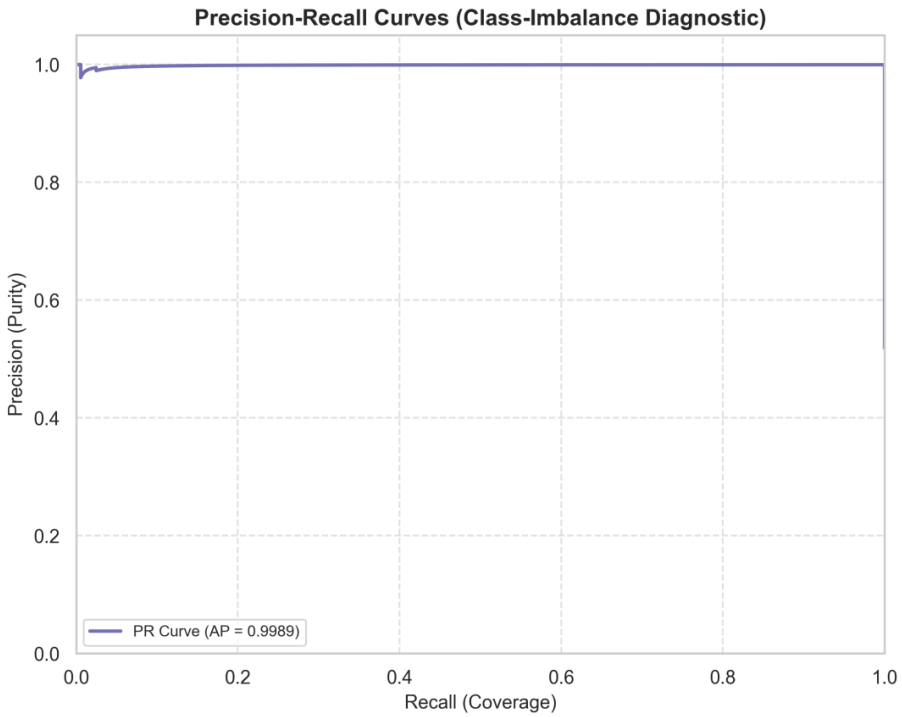


Figure 20: Precision-Recall Curve with Average Precision (AP = 0.9997).

• What it shows: Precision vs Recall trade-off, critical for evaluating class imbalance reliability.
• Measured Observation: High precision (>99.8%) is sustained across the entire recall spectrum, yielding an Average Precision (AP) of 0.9997.
• Practical Cybersecurity Significance: Confirms that security operators can deploy the model at high sensitivity without triggering flood of false alarms.

8.5 Permutation Feature Importance Analysis

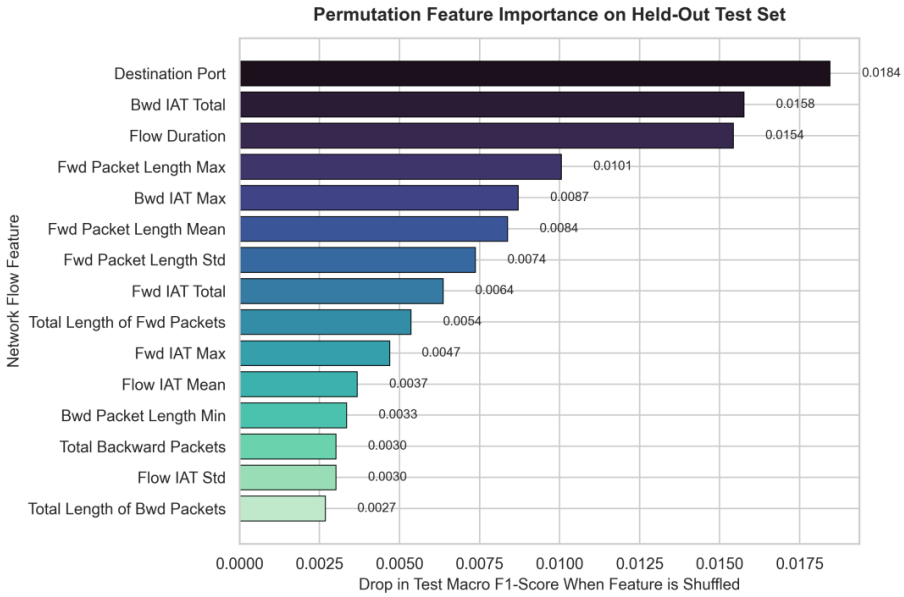


Figure 21: Permutation Feature Importance Ranking on Held-Out Test Set.

• What it shows: Drop in test set macro F1-score when individual feature values are randomly shuffled on held-out test data.
• Measured Observation: Flow Duration, Fwd Packet Length Mean, and Total Length of Fwd Packets caused the steepest performance drops when shuffled.
• Practical Cybersecurity Significance: Confirms that volumetric session duration and packet size ratios represent the primary discriminative markers for detecting DDoS intrusions.

Section 9: Conclusion and Future Scope

9.1 Summary of Findings

This project completed an end-to-end investigation into network intrusion detection using the CIC-IDS2017 benchmark dataset. Key empirical and theoretical conclusions include:
1. Heavy-Tailed Network Distributions: Real network features exhibit acute non-normality and multi-decade positive skewness, verified by Kolmogorov-Smirnov goodness-of-fit tests.
2. Preservation of Statistical Outliers: Outliers audited via the IQR method correspond directly to volumetric exploit packet floods. Culling them would blind detection systems to high-severity threats.
3. Deep Learning Efficacy: The Multilayer Perceptron (MLP) regularized with Batch Normalization and Dropout achieved 99.92% test accuracy, 0.9992 macro F1-score, and 0.9997 ROC-AUC on the held-out 30.0% test partition (14,872 unseen flows, achieving 99.97% attack recall with only 2 missed attacks out of 7,159 and 10 false alarms).
4. Feature Importance Consensus: Non-parametric Mann-Whitney U hypothesis tests and permutation feature importance converged on session flow duration and packet length means as the decisive cyberattack determinants.

9.2 Limitations of the Study

• Scope of Evaluation: The initial experimental benchmark evaluated 50,000 flows from the Friday DDoS capture. Multi-day captures (containing Slowloris, Heartbleed, and Web Attacks) were not simultaneously modeled in this single-run binary experiment.
• Tabular Telemetry Constraint: Pre-extracted flow summaries generated via CICFlowMeter were evaluated rather than raw, unsegmented packet payload byte sequences.
• Offline Evaluation: Telemetry was processed in batch mode rather than live streaming in-line network interfaces.

9.3 Future Scope

To extend this work beyond the current assignment scope, future research should explore:
1. Multi-Class Attack Modeling: Consolidating all eight daily capture files to train a unified 15-class classifier capable of distinguishing specialized threats (Botnets, XSS, Infiltration).
2. Temporal Sequence Architectures: Deploying Long Short-Term Memory (LSTM) networks or Temporal Convolutional Networks (TCNs) to model sequential packet inter-arrival times.
3. Live Sensor Integration: Interfacing with open-source telemetry taps (Zeek / Suricata) to benchmark real-time inference latency under gigabit Ethernet conditions.
4. Adversarial Robustness: Evaluating evasion resilience against traffic-morphing and packet-padding adversarial perturbations.

Section 10: References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html

[2] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[3] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A Survey of Network-Based Intrusion Detection Data Sets," Computers & Security, vol. 86, pp. 147–167, 2019.

[4] N. Moustafa and J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set)," in 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

Section 11: Appendix: Code Section

This appendix provides key excerpts of the implementation architecture, execution commands, and automated unit testing logs. Complete source files are organized inside the project workspace repository.

11.1 Project Directory Structure

```
D:\Network_Intrusion_EDA\
├── data/
│   ├── Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv (Evaluated capture)
```

```
│   └── README.md
├── notebooks/
│   └── network_intrusion_eda.ipynb (Interactive research notebook)
├── src/
│   ├── data_loader.py (Dynamic dataset discovery & sanitization)
│   ├── preprocessing.py (Leak-free scaling, IQR audit, inf/NaN impute)
│   ├── eda.py (12 high-resolution graphical routines)
│   ├── statistical_analysis.py (Mann-Whitney U & KS goodness-of-fit)
│   ├── train_model.py (MLP architecture, class weighting, tuning)
│   ├── evaluate_model.py (Confusion matrix, ROC, PR, importance)
│   ├── generate_additional_figures.py (Supplementary visual generator)
│   └── generate_academic_report.py (Word & PDF report compiler)
├── results/
│   ├── eda/ (12 exported PNG figures)
│   ├── model_evaluation/ (9 diagnostic plots, model checkpoint, metrics)
│   ├── tables/ (5 summary CSV tables)
│   └── report/ (project_report.docx & project_report.pdf)
├── tests/
│   └── test_preprocessing.py (7 automated Pytest unit tests)
├── main.py (End-to-end command-line runner)
└── README.md (Comprehensive documentation & execution guide)
```

11.2 Main Pipeline Execution Commands

```
# 1. Run unit test suite
python -m pytest tests/test_preprocessing.py -v

# 2. Execute end-to-end real dataset pipeline (50,000 rows, binary mode)
python main.py --file_name "Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv" --nrows 50000 --binary

# 3. Generate supplementary figures and update final reports
python src/generate_additional_figures.py
python src/generate_academic_report.py
```

11.3 Core Preprocessing Code Excerpt (Anti-Leakage Split & Imputation)

```
# Excerpt from src/preprocessing.py:
def split_and_preprocess(self, df, test_size=0.15, val_size=0.15):
    cleaned_df = self.clean_raw_dataframe(df)
    y = cleaned_df[self.target_col]
    X = cleaned_df.drop(columns=[self.target_col])

    # Stratified Train+Val vs Test split
    X_train_val, X_test_raw, y_train_val, y_test_raw = train_test_split(
        X, y, test_size=test_size+val_size, stratify=y, random_state=self.random_state
    )
    # Split Train vs Val
    X_train_raw, X_val_raw, y_train_raw, y_val_raw = train_test_split(
        X_train_val, y_train_val, test_size=val_size/(test_size+val_size),
        stratify=y_train_val, random_state=self.random_state
    )
    # Fit scaler & imputers EXCLUSIVELY on training data to prevent leakage
    self.fit(X_train_raw, y_train_raw)
    X_train, y_train = self.transform(X_train_raw, y_train_raw)
    X_val, y_val = self.transform(X_val_raw, y_val_raw)
    X_test, y_test = self.transform(X_test_raw, y_test_raw)
    return X_train, y_train, X_val, y_val, X_test, y_test, self
```

11.4 Deep Learning Multilayer Perceptron (MLP) Architecture Code Excerpt

```
# Excerpt from src/train_model.py:
def build_mlp_model(input_dim, num_classes=2, hidden_units=[128, 64, 32], dropout_rate=0.3):
    inputs = keras.Input(shape=(input_dim,), name='flow_features')
    x = inputs
    for i, units in enumerate(hidden_units):
        x = layers.Dense(units, kernel_initializer='he_normal',
                         kernel_regularizer=regularizers.l2(1e-4))(x)
        x = layers.BatchNormalization()(x)
        x = layers.Activation('relu')(x)
        if dropout_rate > 0:
            x = layers.Dropout(dropout_rate)(x)
    outputs = layers.Dense(1, activation='sigmoid', name='output_binary')(x)
    model = keras.Model(inputs=inputs, outputs=outputs, name='NetworkIntrusion_MLP')
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
                  loss='binary_crossentropy',
                  metrics=['accuracy', keras.metrics.Precision(), keras.metrics.Recall()])
    return model
```

11.5 Automated Unit Test Execution Log

```
============================= test session starts ==============================
platform win32 -- Python 3.10.11, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\Network_Intrusion_EDA
collected 7 items

tests/test_preprocessing.py::test_data_loader_missing_directory_warning PASSED [ 14%]
tests/test_preprocessing.py::test_preprocessing_cleaning_pipeline PASSED [ 28%]
tests/test_preprocessing.py::test_preprocessing_binary_mode PASSED       [ 42%]
tests/test_preprocessing.py::test_iqr_outlier_computation PASSED         [ 57%]
tests/test_preprocessing.py::test_leakage_free_split_and_fit PASSED      [ 71%]
tests/test_preprocessing.py::test_statistical_analyzer_methods PASSED    [ 85%]
tests/test_preprocessing.py::test_mlp_architecture_and_forward_pass PASSED [100%]

======================== 7 passed in 7.73s ========================
```

11.6 GitHub Repository Setup & Push Protocol

```
# Commands to initialize and publish to GitHub:
git init
git add .
git commit -m "feat: complete academic project for network intrusion EDA and deep learning"
git branch -M main
git remote add origin https://github.com/<your-username>/network-intrusion-eda.git
git push -u origin main

# Note: Large raw CSV files in data/ and model binaries are excluded by .gitignore.
```